Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura: Programación Orientada a Objetos

Proyecto: Sistema Bancario en Java

Reporte Técnico Final: Análisis, Diseño e Implementación de un Sistema Bancario (Proyecto 1: Colecciones)

Equipo de Desarrollo:

- Francisco José Coutiño Morales (Núm. Cta: 425121623)
- Ernesto Flamenco Villaseñor (Núm. Cta: 319170599)
- Jaime Erick Torres Nava (Núm. Cta: 322289976)

26 de marzo de 2026

Índice

1. Introducción **2**
1.1. Flujo de Trabajo y Metodología de Desarrollo 2

2. Arquitectura del Modelo de Datos y Lógica Bancaria **2**
2.1. Desacoplamiento de Identidad y Activos Financieros 2
2.2. Evolución a Tipos Primitivos 3

3. Análisis de Estructuras de Datos (Collections) **3**
3.1. Búsqueda Transaccional: Uso de Map (HashMap) 3
3.2. Integridad Financiera: Uso de Set (HashSet) 3
3.3. Auditoría y LIFO: Uso de LinkedList 4

4. Diseño Orientado a Objetos: Archivos y Responsabilidades **4**
4.1. 1. Clase Principal: `SistemaBancario.java` 4
4.2. 2. Clase Controladora: `GestorBanco.java` 4
4.3. 3. La Entidad Contenedora: `Cliente.java` 4
4.4. 4. Productos Activos: `CuentaBasica.java` y `TarjetaCredito.java` 5
4.5. 5. Producto a Plazo: `CuentaInversion.java` 5
4.6. 6. La Auditoría: `Movimiento.java` 5

5. Detección y Corrección de Errores de Lógica ("Bugs") **5**

6. Evolución Arquitectural: Comparativa de Diagramas UML (Inicial vs. Final) **6**
6.1. Diagrama Conceptual Teórico (Versión 1) 6
6.2. Diagrama Arquitectónico Final (Versión As-Built) 7
6.3. Análisis de la Evolución y Cambios Estructurales 7

7. Conclusiones **8**

1. Introducción

El presente documento detalla la arquitectura algorítmica, las decisiones de diseño y el flujo de desarrollo del proyecto "Sistema Bancario El Bienestar". Este sistema fue codificado íntegramente en Java, aplicando los paradigmas de la Programación Orientada a Objetos (POO) [1] y haciendo uso intensivo de la librería *Java Collections Framework* [2] para la gestión transaccional en memoria.

El objetivo fundamental de este proyecto fue modelar las entidades de un entorno financiero real, construyendo un motor de transacciones (Core Banking) capaz de gestionar identidades de clientes, cuentas de captación (débito), instrumentos de rendimiento (pagarés de inversión) y líneas de crédito.

1.1. Flujo de Trabajo y Metodología de Desarrollo

El desarrollo del sistema siguió un enfoque iterativo e incremental, dividido en tres fases principales:

1. **Fase de Diseño Conceptual (V1):** Se diagramó la arquitectura base mediante UML [4] y se seleccionaron teóricamente las estructuras de datos (Maps, Sets y Lists).

2. **Fase de Codificación y Pruebas Unitarias:** Se construyeron las entidades. En esta etapa, el equipo evaluó la posibilidad de integrar una Interfaz Gráfica de Usuario (GUI) utilizando Java Swing o JavaFX. Sin embargo, debido a las restricciones de tiempo del calendario académico y para priorizar la robustez de la lógica de negocio subyacente, la GUI fue descartada de esta iteración. En su lugar, el equipo pivotó hacia la construcción de una Interfaz de Línea de Comandos (CLI) interactiva altamente validada y libre de errores de ejecución.

3. **Fase de Refactorización y Pruebas de Integración (V2 - Final):** Se conectaron los módulos, detectando vulnerabilidades de seguridad financiera que obligaron a reescribir la lógica de validación de fondos, resultando en el diagrama UML definitivo presentado en este informe.

2. Arquitectura del Modelo de Datos y Lógica Bancaria

2.1. Desacoplamiento de Identidad y Activos Financieros

Una de las características más fuertes de la ingeniería de este proyecto es la separación estricta entre el ID del Cliente y los números de cuenta. En sistemas novatos, es común que un usuario sea directamente üna cuenta". Nuestro sistema implementa la realidad bancaria bajo la cardinalidad 1:N:

- **El ID de Cliente (Expediente Base):** Representa a la persona física. Es un número generado aleatoriamente (ej. 10000 a 99999). Sirve como llave de acceso privada interna para el banco (`GestorBanco`).

- **La Cuenta Básica (El Medio Líquido):** Es el producto principal de captación. Su número (ej. rango 100000 - 900000) es público para que terceros realicen depósitos.
- **La Cuenta de Inversión (El Contrato Fijo):** Posee un número de folio completamente distinto (rango 500000+) porque legal y financieramente es un contrato a plazo fijo (28, 91 o 182 días), no una cuenta transaccional. Su separación permite que el dinero esté çongeladoz sujeto a cálculo de interés compuesto, sin mezclarse con el saldo disponible para compras diarias.

Esta normalización de datos asegura que el banco pueda ofrecer escalabilidad: un solo cliente puede tener tres tarjetas de crédito y dos inversiones bajo un mismo paraguas de identidad.

2.2. Evolución a Tipos Primitivos

A diferencia del primer acercamiento, todos los identificadores cambiaron de `String` a `int`. A nivel computacional, comparar cadenas (`String.equals()`) requiere analizar arreglos de caracteres, lo cual tiene un costo de tiempo de Big-O lineal donde la longitud importa. Usar enteros (`int`) reduce las comparaciones en memoria a una operación de CPU directa en tiempo constante, eficientando radicalmente la ejecución de las tablas *Hash*.

3. Análisis de Estructuras de Datos (Collections)

El *Java Collections Framework* proporciona la base para el almacenamiento en memoria. Se seleccionaron tres interfaces distintas resolviendo problemas específicos:

3.1. Búsqueda Transaccional: Uso de Map (HashMap)

Implementado en `GestorBanco` como `Map<Integer, Cliente>directorioClientes`.

- **Justificación Algorítmica:** En un banco, la operación más repetida es buscar el expediente del cliente antes de cualquier retiro o depósito. Un arreglo convencional (`ArrayList`) forzaría una búsqueda lineal de complejidad de tiempo lineal. El `HashMap` [3] utiliza una función de dispersión matemática (*hashing*) sobre el ID entero, encontrando al cliente instantáneamente incluso si el banco tuviera millones de usuarios.

3.2. Integridad Financiera: Uso de Set (HashSet)

Implementado en la clase `Cliente` para almacenar productos: `Set<CuentaBasica>cuentas`.

- **Justificación Algorítmica:** La regla fundamental de bases de datos es evitar duplicidad de llaves primarias. Un `Set` por definición matemática no permite elementos duplicados. Si un fallo del sistema intentara asignarle al cliente un contrato previamente existente, la estructura lo rechaza silenciosamente sin necesidad de programar iteraciones manuales restrictivas.

3.3. Auditoría y LIFO: Uso de LinkedList

Implementado en `CuentaBasica` y `TarjetaCredito` como `LinkedList<Movimiento>historial`.

- **Justificación Algorítmica:** Originalmente se planteó un `ArrayList`, pero un Estado de Cuenta debe mostrar siempre la transacción más reciente en la cima. Esto es el clásico modelo Pila (*Stack*) o Comportamiento **LIFO** (Último en entrar, primero en salir). Usando `LinkedList` y su método nativo `addFirst()`, insertamos objetos en la cabecera rápidamente. Si usáramos una lista estática, Java tendría que desplazar todos los elementos subsecuentes en memoria, lo cual causaría un cuello de botella terrible con historiales largos.

4. Diseño Orientado a Objetos: Archivos y Responsabilidades

El sistema cumple con el Principio de Responsabilidad Única (SRP). A continuación se analizan los componentes y las líneas de código críticas explicadas mediante sus propios comentarios técnicos:

4.1. 1. Clase Principal: `SistemaBancario.java`

Actúa como el hilo conductor (Main) de la aplicación. Aloja el bucle `while` que mantiene activo al Çajero"mediante la clase `Scanner`.

- **Lógica destacada (`sn.nextLine();`):** Tras un método como `sn.nextInt()`, el carácter invisible *Enter* queda flotando en el buffer del teclado. El sistema ejecuta repetidamente este comando de limpieza de buffer como técnica de programación defensiva para evitar que el escáner lance excepciones o salte menús.

4.2. 2. Clase Controladora: `GestorBanco.java`

Es el cerebro operativo. Administra el directorio global y actúa de puente entre el mundo exterior (Main) y las entidades internas.

- **Comentario Técnico del Código:** `// 2. Intentamos retirar el dinero. Si regresa 'true', es que sí tenía saldo`. Esta línea refleja la validación booleana. El gestor no manipula los saldos matemáticamente, delega esa responsabilidad (Encapsulamiento) al objeto `CuentaBasica`.

4.3. 3. La Entidad Contenedora: `Cliente.java`

Almacena los datos personales e instancia colecciones `HashSet` vacías en su constructor.

- **Lógica destacada (`return !cuentas.isEmpty();`):** Una validación elegante para los requisitos del banco. El cliente debe tener cuenta básica para abrir inversiones. La expresión evalúa si la colección no está vacía, retornando la verdad absoluta en una sola línea de código sin requerir condicionales voluminosas.

4.4. 4. Productos Activos: `CuentaBasica.java` y `TarjetaCredito.java`

Ambos extienden la lógica de retiros y depósitos, pero la tarjeta aplica validaciones de límite de endeudamiento (`deudaActual + monto <= limiteCredito`).

- **Comentario Técnico del Código:** `// 2. Guardamos el movimiento al principio de la lista (LIFO)`. Empleado justo después de alterar la variable `saldo`, garantizando que cada cambio de variable se acompañe atómicamente de un registro inmutable en el historial.

4.5. 5. Producto a Plazo: `CuentaInversion.java`

Contiene fórmulas matemáticas y condicionales estrictas para asignar la tasa de interés anual con base en validaciones de plazo de días (28, 91, 182). Retorna montos flotantes calculando rendimiento por días divididos entre el año financiero (360 días).

4.6. 6. La Auditoría: `Movimiento.java`

Es una clase inmutable. Sus variables solo se asignan por constructor.

- **Lógica destacada:** Hace uso de `LocalDateTime.now()` de `java.time`. Abandona el simple formato de fecha e incluye *timestamps* completos (horas, minutos, segundos), esencial para reportes de auditoría pericial bancaria.

5. Detección y Corrección de Errores de Lógica (”Bugs”)

Durante el ciclo de desarrollo se detectaron errores semánticos graves que no detenían la compilación, pero destruían la lógica contable financiera. Estos fueron corregidos en el código As-Built:

1. **Vulnerabilidad de ”Dinero Fantasma”:** En los primeros *builds*, el método `abrirCuentaInversion` simplemente instanciaba el objeto. Esto significaba que el cliente obtenía un instrumento financiero creados ”de la nada”, sin que su saldo principal disminuyera.

2. **Solución Estructural implementada:** Se obligó al sistema a pedir el número de cuenta de fondeo. El método ahora intercepta el flujo, intenta ejecutar `cuentaOrigen.retirar(monto)` Si este método devuelve `false` (sin saldo), el flujo de inversión aborta instantáneamente. Esta misma corrección se aplicó al pago de la Tarjeta de Crédito.

3. **Memory Leaks por Referencias Huérfanas:** Cuando una inversión finalizaba y se pagaba al usuario, el objeto seguía vivo en la memoria del `Set`. Se implementó el comando `remove` para eliminar la referencia exacta, permitiendo que el *Garbage Collector* de Java libere la RAM.

6. Evolución Arquitectural: Comparativa de Diagramas UML (Inicial vs. Final)

El diseño del sistema experimentó una maduración significativa durante la fase de codificación y pruebas. A continuación, se contrastan el modelo conceptual original y la arquitectura definitiva implementada.

6.1. Diagrama Conceptual Teórico (Versión 1)

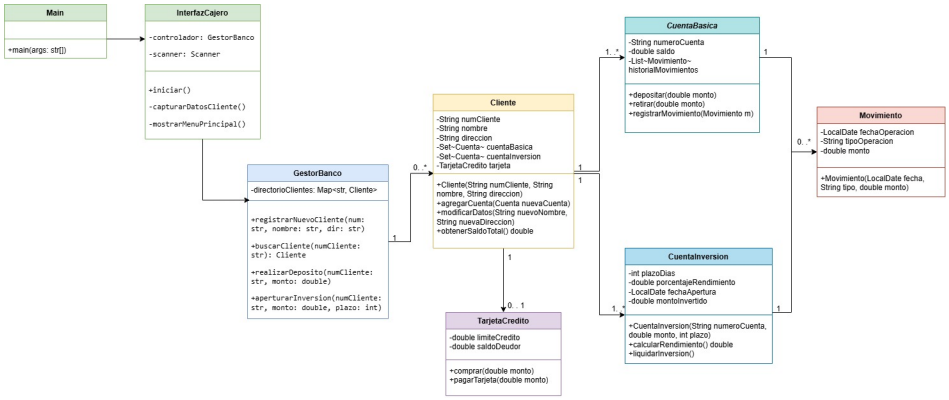

Figura 1: Diseño Preliminar del Sistema Bancario

6.2. Diagrama Arquitectónico Final (Versión As-Built)

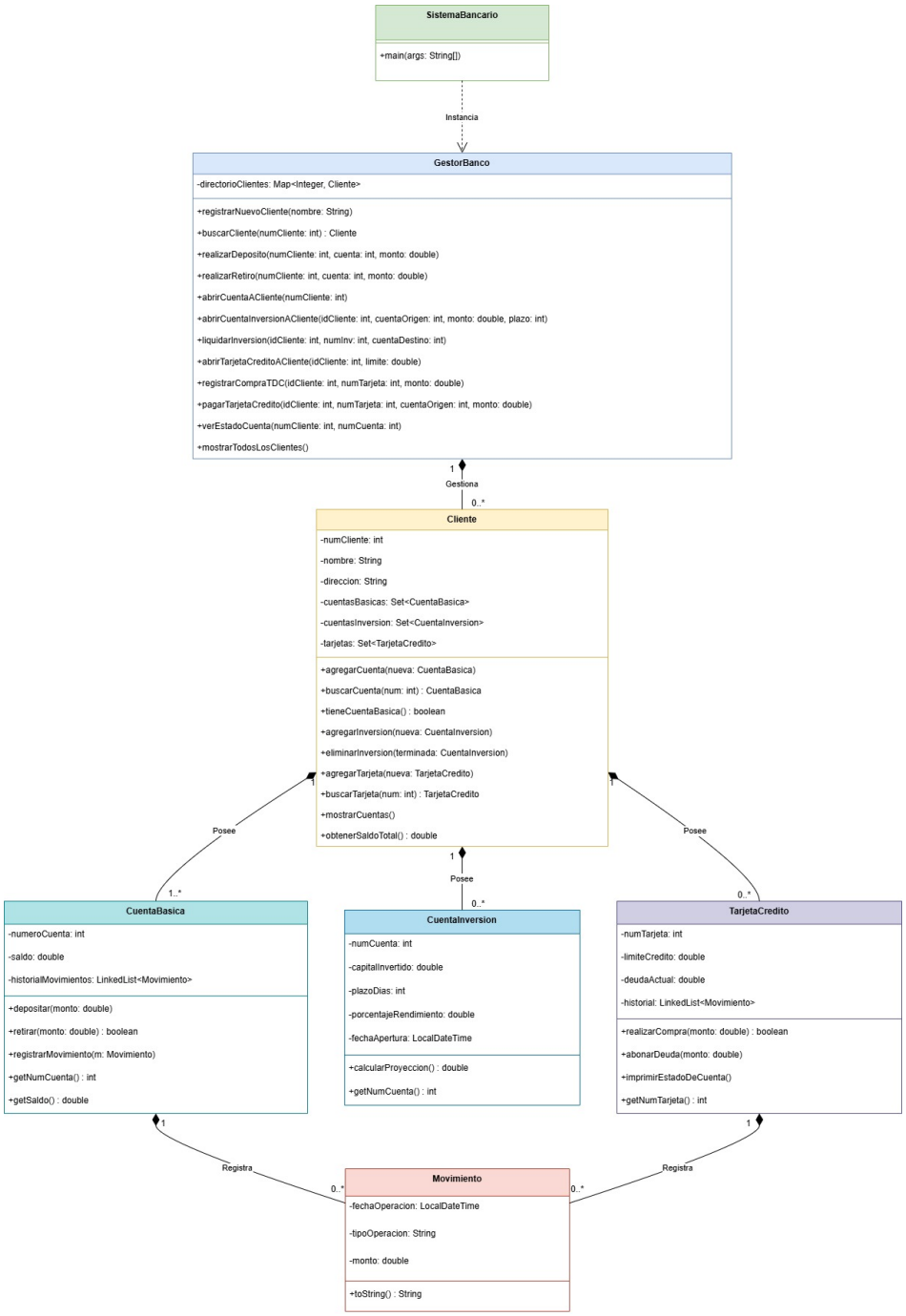

Figura 2: Arquitectura Definitiva e Implementada del Sistema

6.3. Análisis de la Evolución y Cambios Estructurales

Al comparar la Figura 1 con la Figura 2, se evidencian ajustes críticos impulsados por necesidades técnicas reales descubiertas durante el desarrollo:

- **Absorción de la Interfaz Visual:** En el modelo inicial existía una clase `InterfazCajero`. Durante el desarrollo, se comprobó que esta capa añadía un nivel de indirección innecesario para un sistema CLI. Por ello, se fusionó su responsabilidad directamente en la clase `SistemaBancario` (Main), reduciendo el acoplamiento y centralizando el uso de la clase `Scanner`.

- **Corrección de la Multiplicidad del Crédito:** El diagrama original establecía una relación 0.,1 entre Cliente y Tarjeta de Crédito (limitando al usuario a poseer una sola tarjeta). En la versión final, esto se corrigió a 0..∗ (cero a muchas), alojadas dentro de un `Set<TarjetaCredito>`, reflejando la realidad comercial bancaria.

- **Trazabilidad de Historial Completo:** Originalmente, solo la Cuenta Básica generaba registros de `Movimiento`. En la implementación final, se trazó una nueva línea de agregación (0..∗) desde `TarjetaCredito` hacia `Movimiento`, obligando al sistema a auditar también las compras a crédito y abonos de deuda.

- **Refactorización de Firmas y Retornos:** El diagrama final muestra cómo los métodos de `GestorBanco` mutaron. Por ejemplo, la creación de inversiones pasó de recibir solo un monto, a exigir un `cuentaOrigen: int`. Este cambio en el diseño fue la respuesta directa para parchar la vulnerabilidad de "dinero fantasma.^ex^plicada en la Sección 5.

7. Conclusiones

El desarrollo del "Sistema Bancario El Bienestar" es un caso de estudio exitoso en la aplicación del *Collections Framework* y los algoritmos de validación asimétrica. Se comprobó que una correcta abstracción en la Programación Orientada a Objetos permite simular normativas financieras estrictas del mundo real. La transición documentada desde el diagrama UML preliminar hasta el modelo As-Built final demuestra un proceso iterativo maduro. El uso orquestado de estructuras de almacenamiento complejas, como `HashMap` y listas doblemente enlazadas, otorga al sistema un nivel de solidez computacional listo para su evaluación académica.

Bibliografía

[1] Oracle. (s.f.). *Lesson: Object-Oriented Programming Concepts*. The Java™ Tutorials. Recuperado de `https://docs.oracle.com/javase/tutorial/java/concepts/index.html`

[2] Oracle. (s.f.). *Lesson: Introduction to Collections*. The Java™ Tutorials. Recuperado de `https://docs.oracle.com/javase/tutorial/collections/intro/index.html`

[3] Oracle. (s.f.). *Class HashMap¡K, V¿*. Java Platform, Standard Edition 8 API Specification. Recuperado de `https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html`

[4] draw.io. (s.f.). *UML Class Diagrams*. draw.io blog. Recuperado de `https://www.drawio.com/blog/uml-class-diagrams`